Recall on AWS — Cloud Computing Project Report

Oct 19, 2026

Recall turns a team's terminal history into shared, searchable knowledge, and we deployed it on AWS with one CloudFormation template: EC2, Amazon RDS for PostgreSQL, HTTPS, CI/CD and demo data, live in about 9 minutes.

Problem and solution

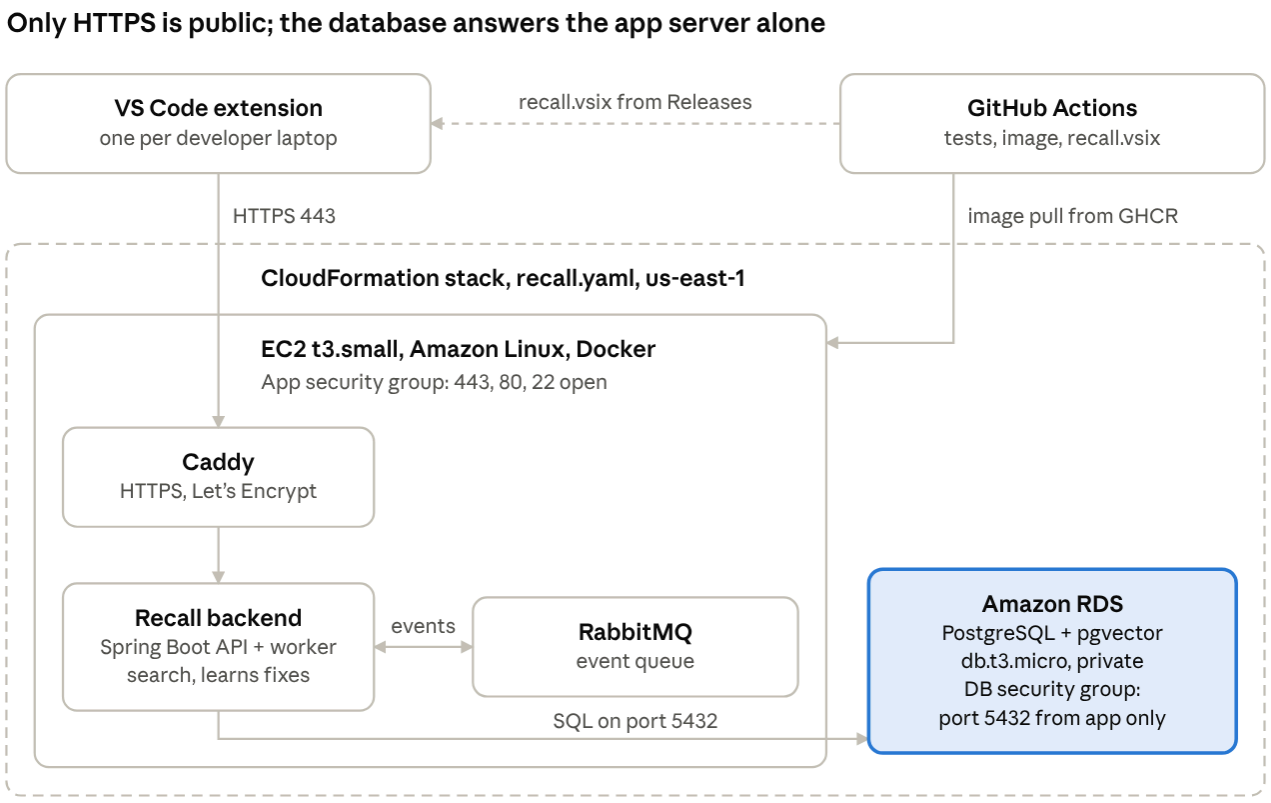
Developers on a team keep solving the same terminal problems again: a command they half remember, an error a teammate already fixed last week. That knowledge sits in individual shell histories and is lost to everyone else.

Recall makes it shared. A VS Code extension records each terminal command (secrets masked first), and a cloud backend stores the team's history and answers two questions:

- **Search by meaning.** Typing *undo my last commit but keep the changes* returns `git reset --soft HEAD~1`, with who used it and when.
- **Fixes for known errors.** When a command fails, Recall checks whether a teammate hit the same error and offers the command that fixed it.

Nobody writes documentation: when a failed command is followed by a related command that succeeds, the backend learns that command as the team's fix. Each team sees only its own history.

Architecture



Recall on AWS: one CloudFormation stack, EC2 + RDS

VS Code talks to Caddy over HTTPS; Caddy forwards to the backend, which queues events in RabbitMQ and stores and searches them in Amazon RDS. GitHub Actions builds the image the server pulls and the extension developers install.

AWS services used

The whole system runs in the AWS Academy Cloud Developing sandbox (us-east-1), within its limits of small instance types and no IAM user creation.

Service	Configuration	Role in Recall
AWS CloudFormation	One template, `deploy/aws/recall.yaml`, 10 parameters with defaults	Creates every resource below in one step; completes only after Recall passes its health check
Amazon EC2	t3.small (2 vCPU, 2 GB), Amazon Linux 2023, 20 GB gp3, `vockey` key, `LabInstanceProfile`	Runs the backend, RabbitMQ and Caddy as Docker containers

Service	Configuration	Role in Recall
Amazon RDS for PostgreSQL	db.t3.micro, 20 GB gp2, single-AZ, not publicly accessible, pgvector extension	Stores teams, commands, errors and learned fixes; vector search by meaning
Amazon VPC security groups	App group: 443, 80, 22 inbound. DB group: 5432 only from the app group	Network isolation: the database is unreachable from the internet
AWS Systems Manager (Session Manager)	Through `LabInstanceProfile`	Browser terminal on the server without opening SSH keys
GitHub Actions + GitHub Container Registry	CI on every push; Release workflow after CI passes	Builds and tests the code, publishes the backend image and the `recall.vsix` extension

HTTPS uses a free Let's Encrypt certificate issued by Caddy for the instance's `sslip.io` name (for example `3-80-65-109.sslip.io`), so no domain purchase is needed.

Cloud computing concepts applied

Concept	Where it shows in Recall
Infrastructure as Code (IaC)	The CloudFormation template recreates the full system in every sandbox session, identically, in about 9 minutes
Infrastructure as a Service (IaaS)	EC2 virtual machine rented on demand
Platform / managed services	Amazon RDS runs PostgreSQL: AWS handles provisioning, patching and storage
Containerisation	Backend, RabbitMQ and Caddy run as Docker containers from `deploy/aws/docker-compose.yml`
CI/CD	GitHub Actions runs all tests on each push, then builds and publishes the container image and the extension
Asynchronous messaging	The API puts incoming events on a RabbitMQ queue and answers at once; a worker stores and learns from them; failed batches go to a dead-letter queue after 3 attempts

Concept	Where it shows in Recall
Multi-tenancy	Many teams share one server; the team id comes only from the signed login token (JWT), so no team can read another's data
Network security and least privilege	Security groups expose only HTTPS publicly; the database accepts connections only from the app's security group
Encryption in transit	HTTPS with an automatically issued Let's Encrypt certificate
Health checks and self-healing	`/api/v1/health` reports database, queue and model status; Docker restarts failed containers; CloudFormation waits for a healthy signal
Fault tolerance at the edge	The extension queues commands offline and retries with backoff (1 s to 60 s), so nothing is lost while the server is down
Stateless API	Login is a signed token, not a server session, so more backend instances could be added behind a load balancer

Deployment

The sandbox deletes every resource when its session ends, so deployment is designed to be repeated: one upload, about 9 minutes, no manual server setup.

1. **Start the lab.** AWS Academy Cloud Developing → Sandbox → Start Lab, then open the AWS Console in us-east-1.
2. **Create the stack.** CloudFormation → Create stack → upload `recall.yaml` → stack name `recall` → defaults → Submit.
3. **CloudFormation builds the infrastructure.** Security groups (about 1 minute), then the RDS database (about 6 minutes), then the EC2 instance.
4. **The instance configures itself** (about 3 minutes, `deploy/aws/setup.sh`):
   a. installs Docker and adds 2 GB swap;
   b. pulls the prebuilt backend image from GitHub Container Registry;
   c. starts the backend, RabbitMQ and Caddy, connected to RDS;
   d. loads a demo team (2 teammates, about 200 commands, 15 learned fixes);
   e. gets the HTTPS certificate and writes a landing page with the URL and invite code;
   f. signals CloudFormation, which marks the stack CREATE_COMPLETE.

5. **Connect VS Code.** Install `recall.vsix` from the GitHub Releases page, run *Recall: Set server URL* with the stack's `RecallUrl` output, then *Recall: Sign in*.

Secrets are never typed by hand: the database password is derived from the stack's unique id and the login-token secret is generated on the server.

Evidence from the live deployment

All screenshots were taken on 19 October 2026 in the AWS Academy Cloud Developing sandbox, while the stack was running.

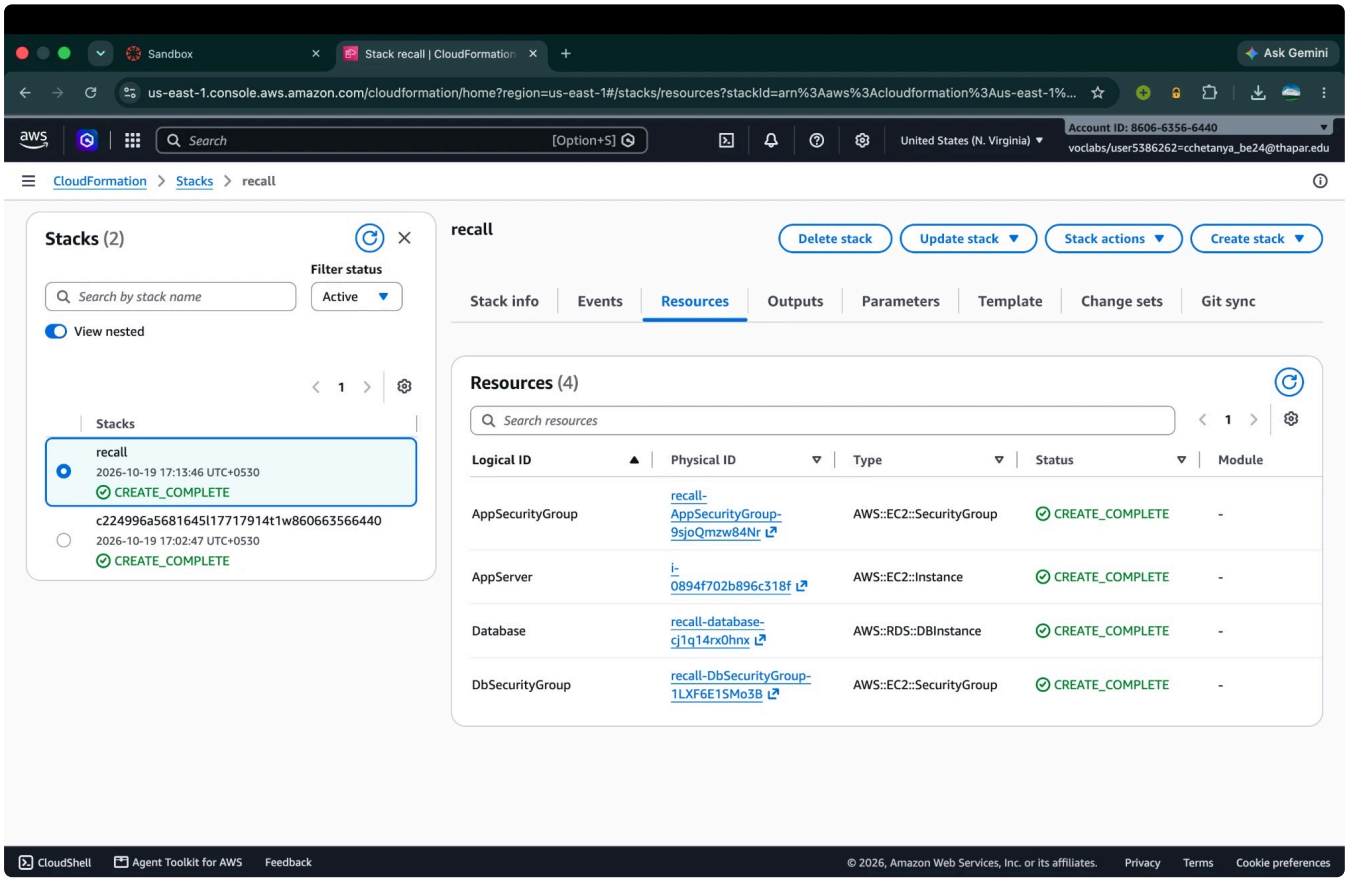


Figure 1. CloudFormation created all four resources. Two security groups, the RDS database and the EC2 instance, all CREATE_COMPLETE.

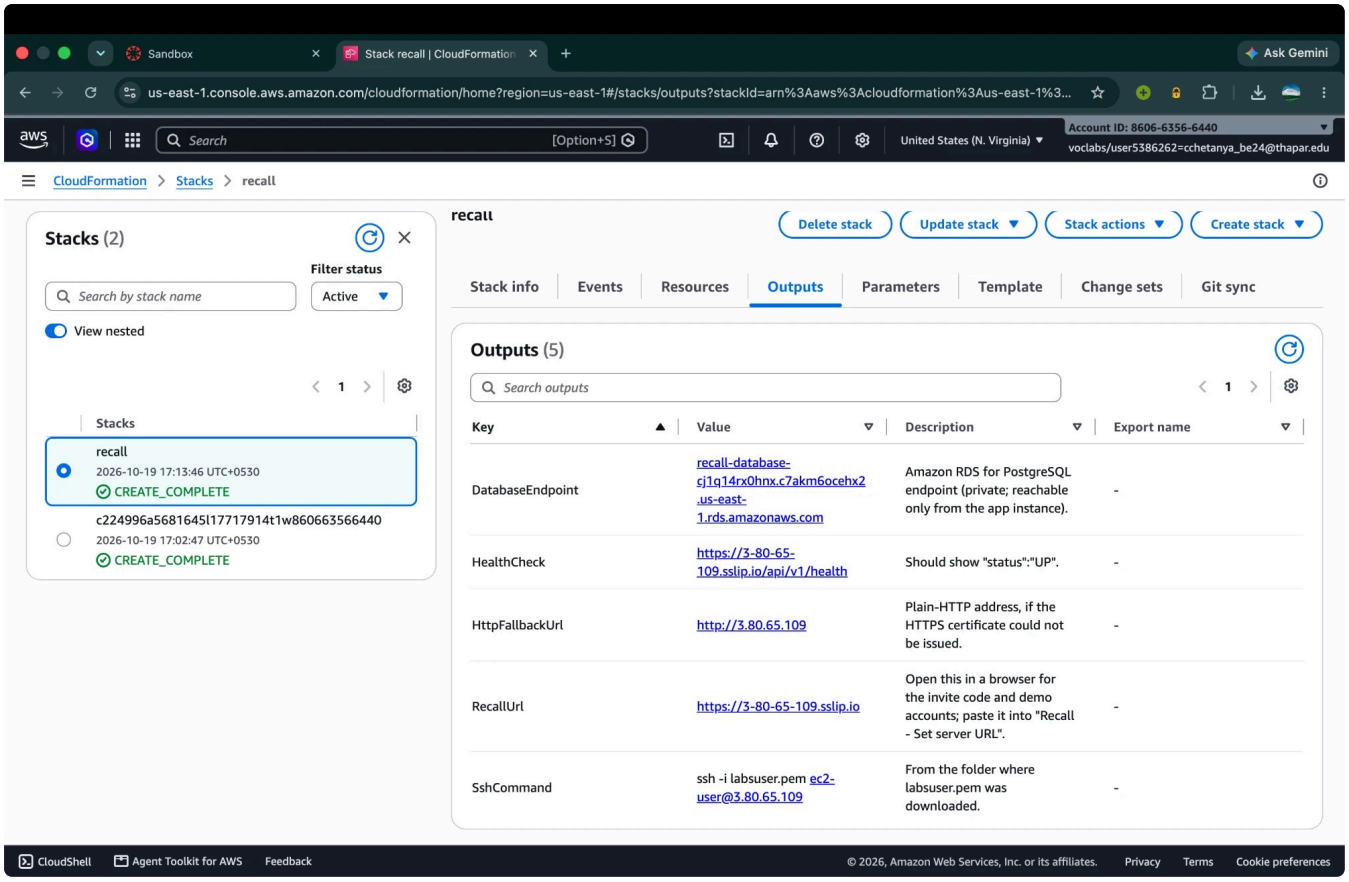


Figure 2. Stack outputs. The public Recall URL over HTTPS, the health-check URL and the private database endpoint.

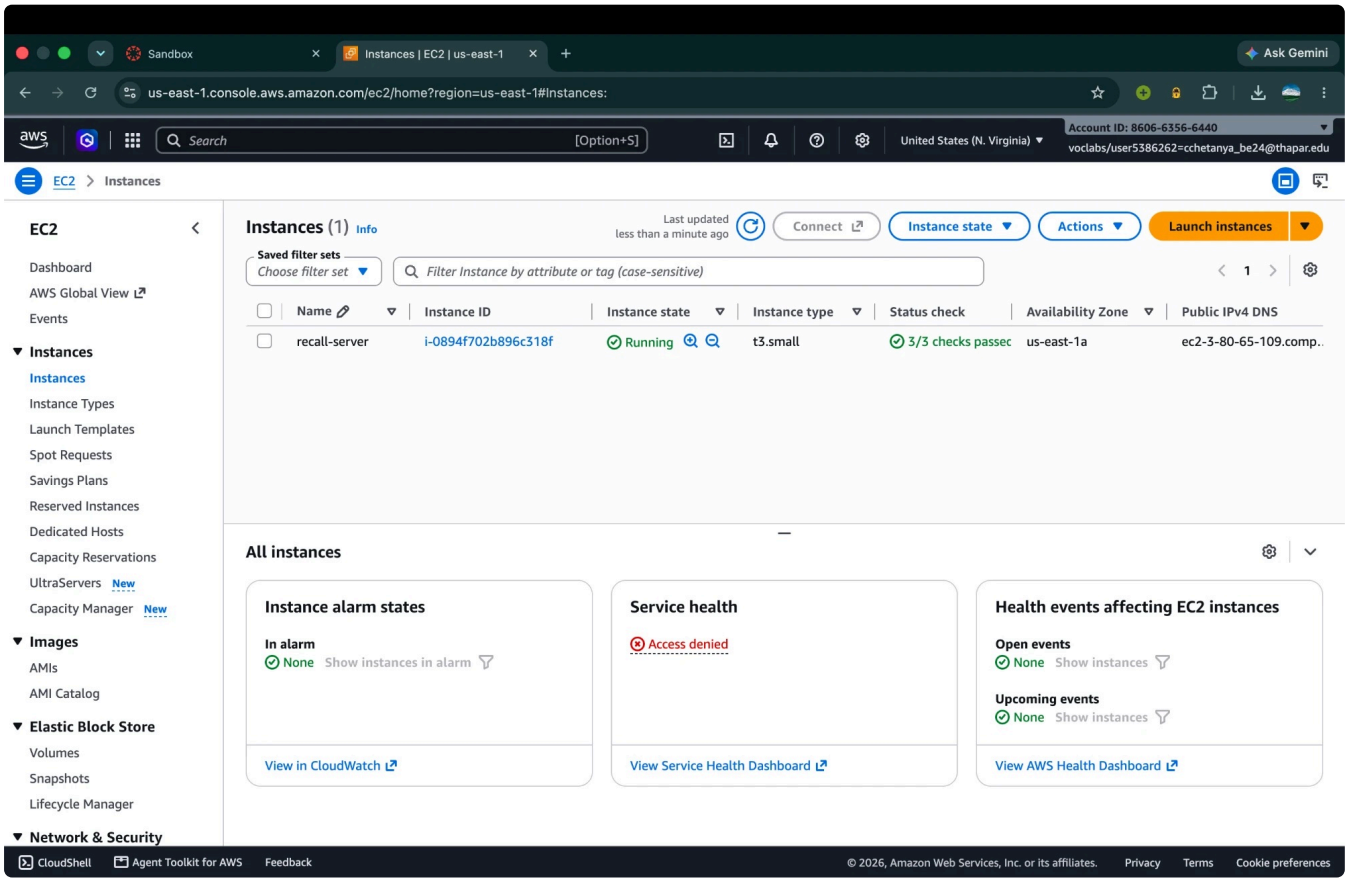


Figure 3. The EC2 instance. `recall-server`, t3.small, running, 3/3 status checks passed.

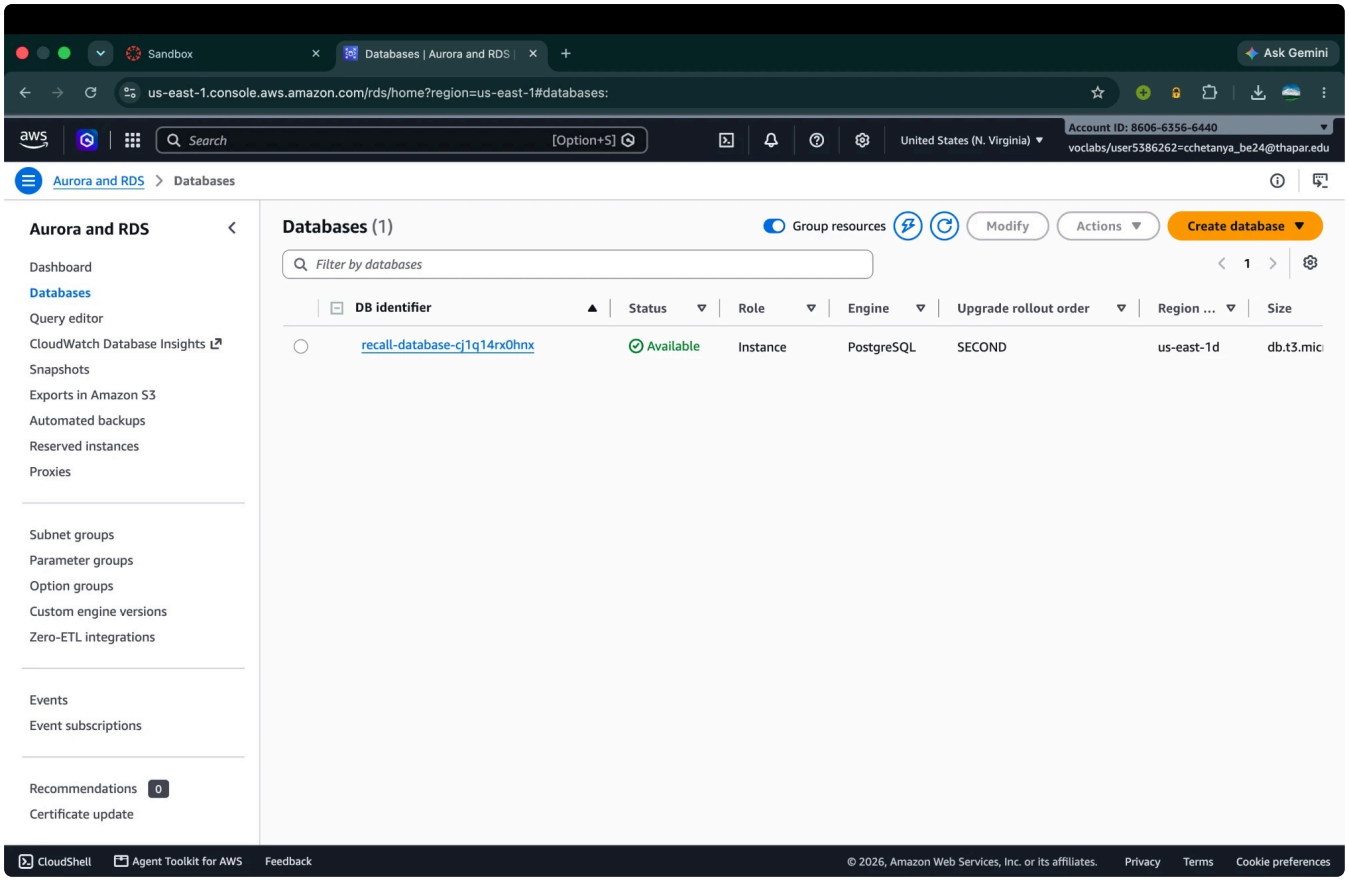


Figure 4. The managed database. Amazon RDS for PostgreSQL, db.t3.micro, available.

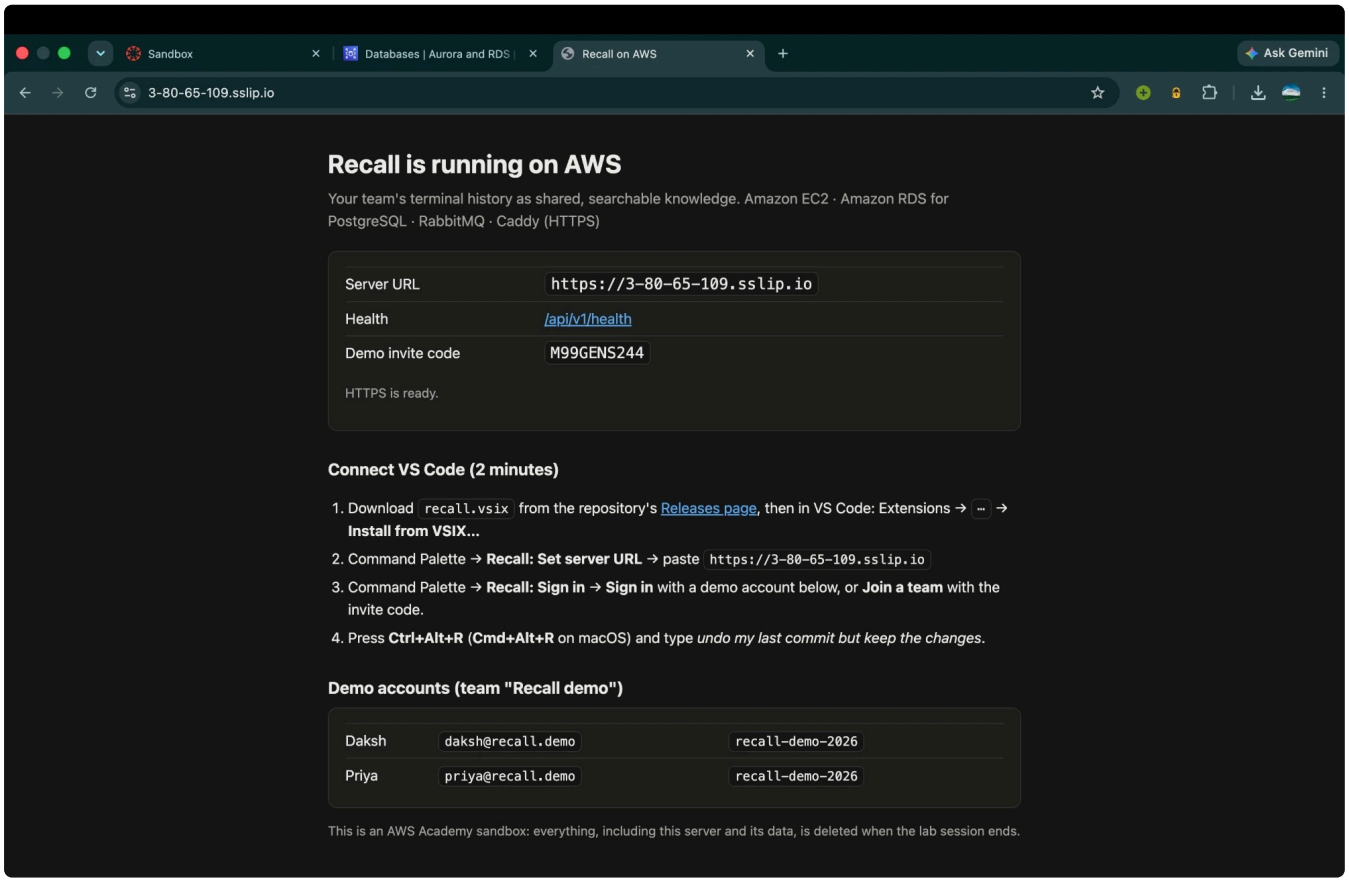


Figure 5. Recall live on the internet. The landing page served over HTTPS by Caddy, with the server URL and the demo invite code.

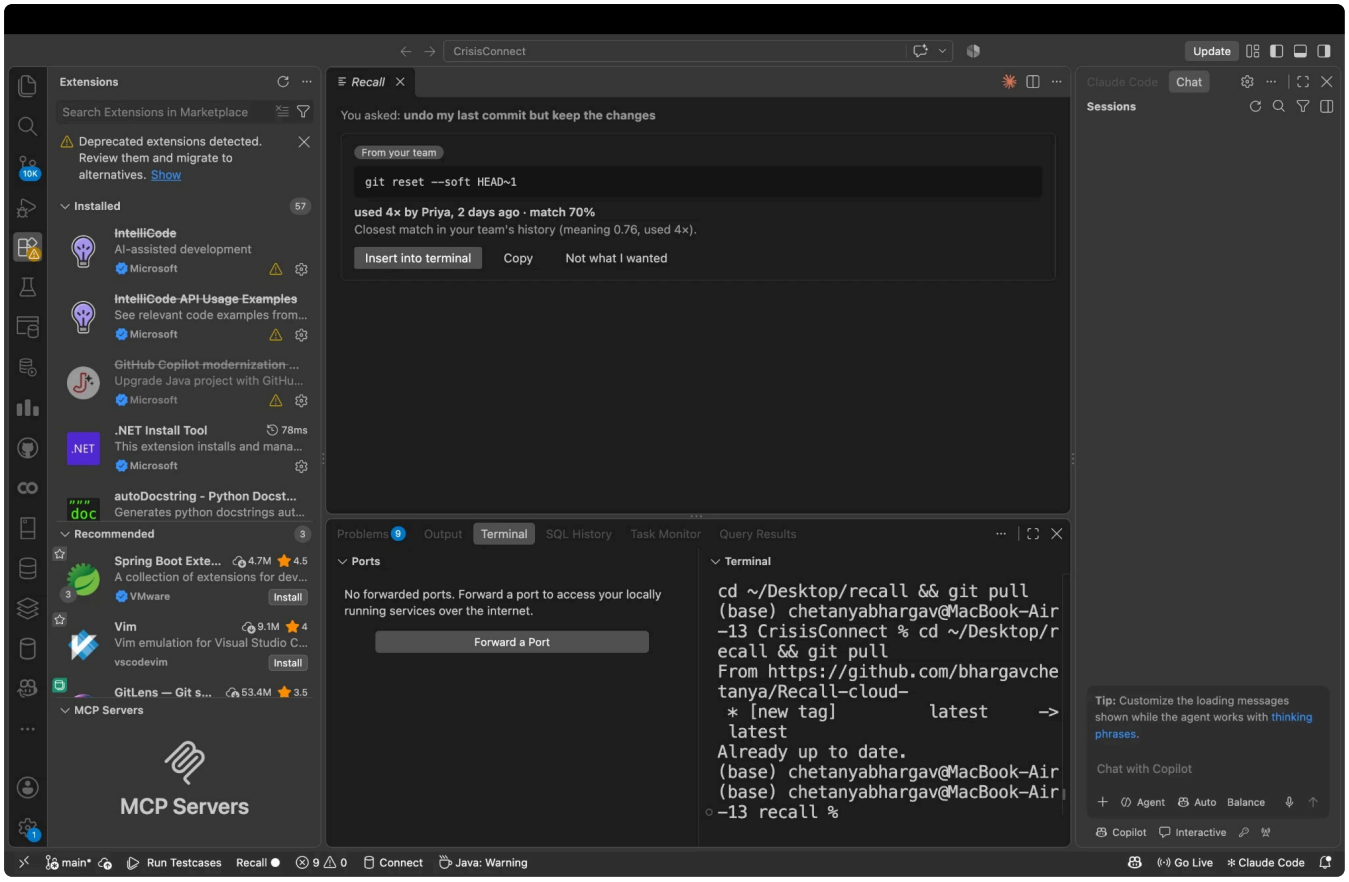


Figure 6. Search by meaning. A plain-English question returns `git reset --soft HEAD~1` from the team's history, with who used it and when.

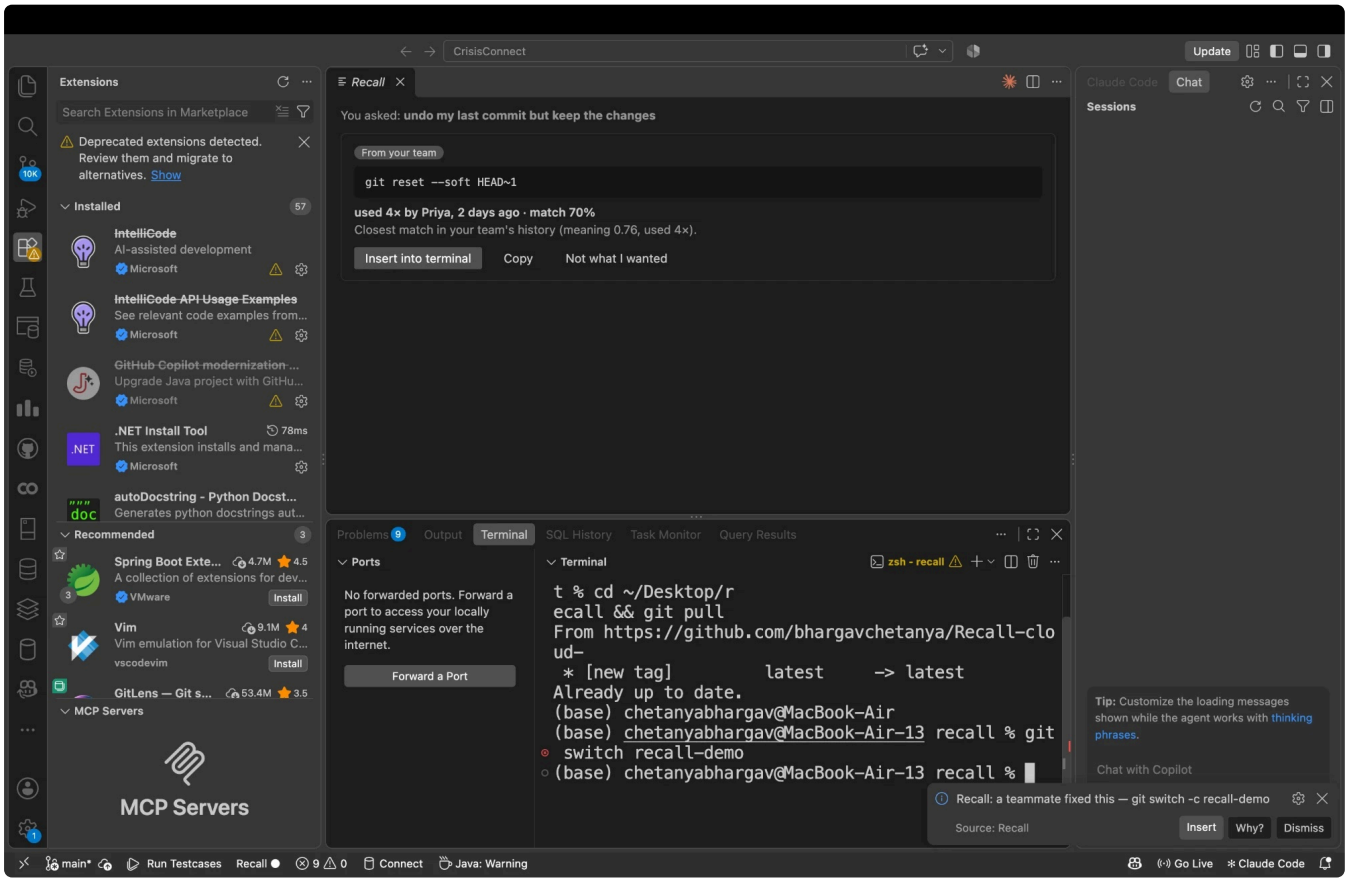


Figure 7. A failed command triggers a team fix. After `git switch recall-demo` fails, Recall suggests the command a teammate used to fix the same error.

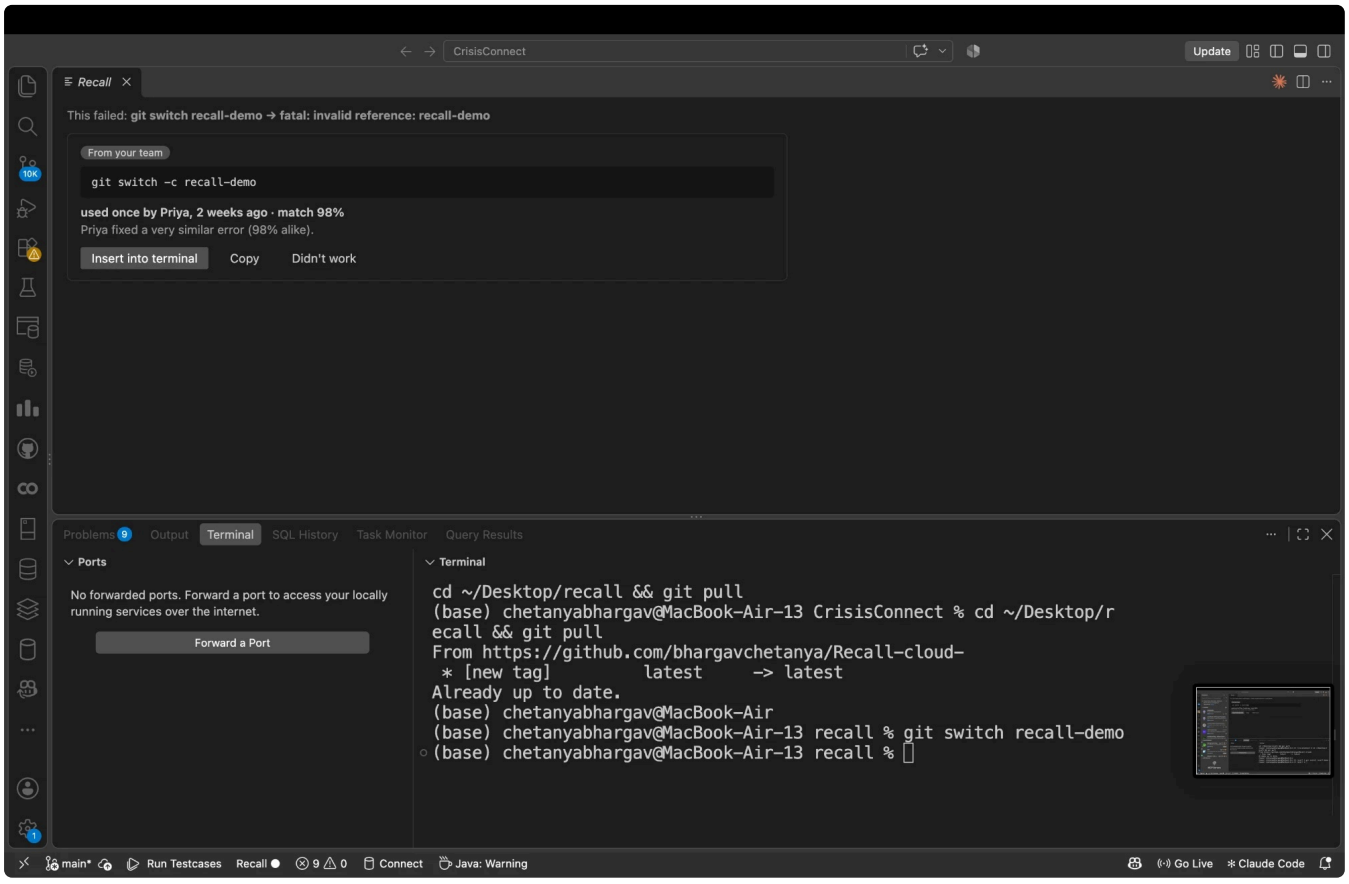


Figure 8. Why this fix. The fix comes from Priya's history and matches this error at 98%.

Testing and results

Every push to GitHub runs the full test suite in GitHub Actions, and a release is published only after it passes. The live AWS deployment then behaved exactly as the tests predict.

Level	What it covers	Result
Backend acceptance tests (Testcontainers: real PostgreSQL + pgvector and RabbitMQ)	T1 search hit, T2 search miss, T3 known error, T4 learning loop, T5 team isolation, T6 secret masking, T7 error fingerprints, T8 burst of 1 000 events, T9 feedback demotion, T10 latency	Pass in CI

Level	What it covers	Result
Extension unit tests (Vitest)	Masking, offline queue, API client, status bar, menu, server URL handling (153 tests)	Pass in CI
Extension integration tests	A real VS Code terminal sending commands to a test backend	Pass in CI
End-to-end script (`scripts/e2e.sh`)	15 checks against a running server: sign-up, search, learned fix, masking, team isolation	E2E PASS in CI and through the HTTPS proxy
Live deployment on AWS, 19 October 2026	The CloudFormation stack in the Cloud Developing sandbox	CREATE_COMPLETE in 9 minutes; health UP for database, queue and model

Live demo results on AWS:

- Search *undo my last commit but keep the changes* → `git reset --soft HEAD~1`, used 4 times by Priya, match 70%.
- Running `git switch recall-demo` failed with *fatal: invalid reference* → Recall offered `git switch -c recall-demo`, the fix Priya found, 98% match.
- The database endpoint is private: only the app's security group can reach port 5432.

Limitations and future work

The main limit comes from the sandbox, not the design: everything is deleted when a session ends, so team data does not persist and the server address changes each time.

Limitation	Next step on a full AWS account
Data is lost at the end of each sandbox session	Keep the stack running; enable RDS automated backups and snapshots
One EC2 instance is a single point of failure	Application Load Balancer + Auto Scaling group across two Availability Zones (the API is already stateless)
RabbitMQ runs in a container on the same instance	Amazon MQ, or Amazon SQS with a dead-letter queue

Limitation	Next step on a full AWS account
The URL changes every session	A Route 53 domain and an Elastic IP, with an ACM certificate on the load balancer
No central logs or alerts	CloudWatch logs, metrics and alarms on health and latency
Optional AI suggestions (Ollama) need more memory than the sandbox allows	A larger instance or a managed model API for questions the team has not answered

Within these limits, the project shows the core cloud ideas end to end: infrastructure built from code, a managed database, containers, CI/CD, network isolation and a working multi-team service over HTTPS.